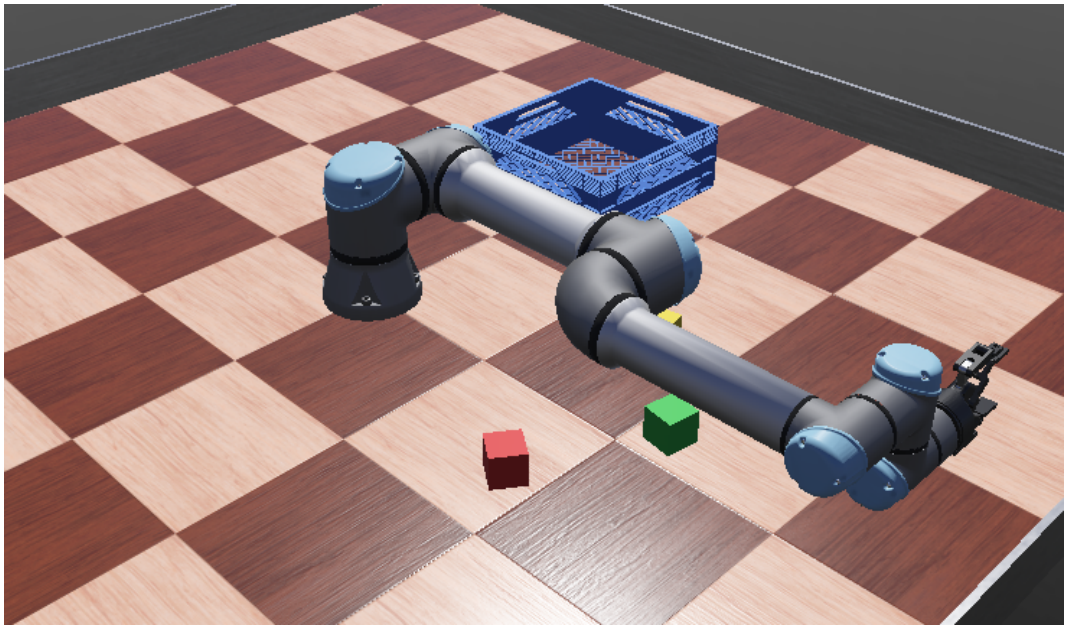
Zadanie 2: Pick and place



Opis zadania

Przed Tobą ramię robotyczne **Universal Robots UR5e** z chwytakiem **Robotiq 2F-85**. Na podłodze leżą trzy kostki (czerwona, zielona i żółta). Napisz program, który przeniesie je do niebieskiej skrzynki.

Świat: `worlds/pick_and_place.wbt`.

Struktura folderu

```
02_pick_and_place/
├── worlds/pick_and_place.wbt
└── controllers/
    ├── pick_and_place/
    │   ├── pick_and_place.py      TWÓJ PROGRAM
    │   └── ur5e_kinematics.py     kinematyka ramienia (nie musisz zmieniać)
    └── judge/judge.py             sędzia (nie modyfikuj)
```

Sterowanie ramieniem

Nie musisz liczyć kątów przegubów. W szkielecie jest gotowa funkcja, która przyjmuje **położenie i orientację chwytaka**, a kąty przegubów oblicza sama (kinematyka odwrotna, plik `ur5e_kinematics.py`):

```
move_to_pose(x, y, z, roll=0, pitch=0, yaw=0, linear=False)
```

- `x, y, z` to położenie punktu chwytu (między końcówkami palców) w metrach. Układ współrzędnych ma początek u podstawy robota na podłodze: oś X do przodu, Y w lewo, Z do góry.
- `roll, pitch, yaw` to orientacja chwytaka w radianach. Dla `roll = pitch = yaw = 0` chwytak jest skierowany pionowo w dół, a palce zamykają się wzdłuż osi X. `yaw` obraca chwytak wokół osi pionowej, a `roll` i `pitch` go przechylają. Do tego zadania zwykle wystarczy sam `yaw`.

- `linear=True` sprawia, że chwytak jedzie po linii prostej (w krokach co 1 cm). Przydaje się przy zjeżdżaniu do kostki i podnoszeniu jej, bo wtedy palce nie zahaczają o kostkę.
- Funkcja zwraca `False`, jeśli pozycja jest poza zasięgiem ramienia.

Inne gotowe funkcje: `move_to(kąty)` (bezpośrednie ustawienie 6 przegubów), `open_gripper()`, `close_gripper()`, `wait(sekundy)`, `current_position()` (aktualne położenie punktu chwytu) i `current_angles()`.

Słownik `CUBES` zawiera położenie środka każdej kostki i jej obrót wokół osi pionowej (`yaw`). Kostki mają bok 5 cm. Jeśli podasz chwytakowi ten sam `yaw` co kostce, palce będą równoległe do jej ścianek. Środek skrzynki jest w `CRATE_CENTER`, a jej ścianki mają 12 cm wysokości.

Szkielet zawiera już przykład: przejazd 15 cm nad czerwoną kostkę.

Zasady i ocena

Sędzia co sekundę sprawdza, ile kostek leży w skrzynce. Ocena kończy się, gdy wszystkie kostki są w skrzynce albo po 120 s czasu symulacji. Liczy się liczba przeniesionych kostek, a przy remisie czas.

Poziomy:

- **podstawowy**: przenieś czerwoną kostkę,
- **pełny**: przenieś wszystkie trzy kostki,
- **dla chętnych**: program działa dla dowolnego położenia kostek i jest jak najszybszy. Przy ocenie tego poziomu zmienimy położenie i obrót kostek (w świecie i w słowniku `CUBES`), więc nie wpisuj współrzędnych na sztywno. Dla porównania nasze rozwiązanie przenosi wszystkie kostki w ok. 21 s.

Wskazówki

- Typowa sekwencja dla jednej kostki: nad kostkę, zjazd w dół, zamknięcie chwytaka, podniesienie, przejazd nad skrzynkę, otwarcie chwytaka.
- Chwytaj kostkę trochę powyżej jej środka (np. 1 cm), żeby palce nie uderzały w podłogę.
- Przy przejeździe nad skrzynkę trzymaj kostkę wysoko (np. 30 cm), żeby nie zahaczyć o ściankę.
- Kostki są symetryczne, więc obrót o 90° (`yaw ± π/2`) też jest dobry. Czasem pozwala to uniknąć dużego obrotu nadgarstka.
- Jeśli chcesz zrozumieć, jak działa kinematyka odwrotna, zajrzyj do `ur5e_kinematics.py`. Rozwiązanie numeryczne (metoda tłumionych najmniejszych kwadratów) jest tam opisane w komentarzach.

Co oddać

- plik sterownika (`.py`) z Twoim rozwiązaniem,
- krótkie nagranie ekranu (do 2 minut) pokazujące przejazd/działanie robota i wynik sędziego w konsoli,
- kilka zdań opisu: jak działa Twoje rozwiązanie, co sprawiało trudność i co byś poprawił(a), mając więcej czasu.

Rozwiązanie wyślij w sposób podany przez organizatorów rekrutacji. Możesz korzystać z dokumentacji, internetu i narzędzi AI, ale na rozmowie poprosimy Cię o wyjaśnienie

swojego kodu, więc upewnij się, że rozumiesz każdą jego linijkę.

Nie modyfikuj plików sędziego ani świata (poza eksperymentami na własny użytek). Przy ocenie uruchomimy Twój sterownik w oryginalnych światach.

Webots: co to jest i jak zacząć

Czym jest Webots

Webots to darmowy, otwartoźródłowy symulator robotów rozwijany przez firmę Cyberbotics. Pozwala zbudować wirtualny świat z robotami, czujnikami i obiektami, a potem sterować robotami programami (tzw. kontrolerami) napisanymi m.in. w Pythonie, C lub C++. Symulacja uwzględnia fizykę: tarcie, masy, zderzenia i szum czujników, więc program, który działa w Webots, zachowuje się podobnie jak na prawdziwym robocie. Dzięki temu możesz uczyć się robotyki bez sprzętu.

Instalacja

1. Pobierz **Webots R2025a** ze strony https://cyberbotics.com (przycisk Download) albo z https://github.com/cyberbotics/webots/releases. Zadania przygotowano i przetestowano w tej wersji.
2. Zainstaluj program jak zwykłą aplikację (Windows: instalator `.exe`, macOS: plik `.dmg`, Linux: pakiet `.deb` lub `snap`).
3. Zainstaluj **Python 3** (https://www.python.org). Na Windows zaznacz w instalatorze opcję „Add Python to PATH”.
4. Jeśli Webots nie znajdzie Pythona sam, wskaż go w *Tools* → *Preferences* → *General* → *Python command* (np. `python` albo pełna ścieżka do `python.exe`).

Pierwsze uruchomienie zadania

1. Pobierz to repozytorium (`git clone` albo *Code* → *Download ZIP*) i rozpakuj je w zwykłym folderze, np. `C:\robotyka\`. **Nie** umieszczaj go w folderze instalacji Webots (`C:\Program Files\Webots`) ani w folderze synchronizowanym z OneDrive.
2. W Webots wybierz *File* → *Open World* i otwórz plik `.wbt` z folderu `worlds` wybranego zadania.
3. Przy pierwszym otwarciu Webots pobiera modele robotów z internetu, więc potrzebne jest połączenie z siecią. Może to potrwać chwilę.
4. Symulację uruchamiasz przyciskiem play (▶), zatrzymujesz przyciskiem pauzy, a przycisk reset (strzałka zawracająca) przywraca stan początkowy.
5. Komunikaty programów, w tym wynik od sędziego, pojawiają się w konsoli na dole okna.

Jak edytować program robota

Kod sterownika znajduje się w folderze `controllers/<nazwa>/<nazwa>.py`. Możesz go edytować we wbudowanym edytorze Webots (prawy klik na robota → *Edit controller*) albo w dowolnym edytorze, np. VS Code. Po zapisaniu zmian zresetuj symulację, żeby program uruchomił się od nowa.

Przydatne źródła:

- dokumentacja Webots: https://cyberbotics.com/doc/guide/index
- opis funkcji (API) dla Pythona: https://cyberbotics.com/doc/reference/index (przełącz język przykładów na Python)
- tutoriale dla początkujących: https://cyberbotics.com/doc/guide/tutorials

Typowe problemy

- **„Skipped unknown … node or PROTO”** lub **„changed by fallback mechanism”**: Webots nie może pobrać modeli. Sprawdź połączenie z internetem i to, czy projekt nie leży w folderze instalacji Webots.
- **Sterownik nie startuje, błąd Pythona**: sprawdź ustawienie *Python command* w preferencjach.
- **Zmiany w kodzie nie działają**: zapisz plik i zresetuj symulację.
- **Symulacja działa wolno**: wyłącz cienie i antyaliasing w *Tools* → *Preferences* → *OpenGL* albo użyj trybu szybkiego (przycisk z podwójną strzałką, *Fast*).